

## UNIVERSITY OF CLAUDE · FREE RESOURCE MAP

# Resource Map

*CSCE 713 Software Security — module-by-module mapping onto free university courseware*

## HOW TO USE THIS

This is your lecture hall. For each module: watch or read the mapped external material first, then work the module slides and notes, which are written to assume you have seen it.

Where a specific lecture number is given, that is the one that covers this module. Where a book chapter is given, read that chapter, not the whole book.

Everything here is free. Resources remain under their own licences — link to them, do not redistribute them.

## Primary sources for the whole course

| Resource                                                                           | Role in this course                                                                                                                                                                                          |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dowd, McDonald & Schuh — The Art of Software Security Assessment                   | <b>The primary reference for Modules 02 through 06 and 11.</b> Dated in its examples and unmatched in its treatment of how defects arise and how to find them by reading. Library copy.                      |
| Zeller et al. — The Fuzzing Book (free, interactive)                               | <b>Module 08's primary source, free and executable.</b> Builds fuzzers from scratch in the browser, which is the right way to understand what coverage guidance actually does.                               |
| Møller & Schwartzbach — Static Program Analysis (free PDF)                         | <b>Module 07's theory, free.</b> The lattice and fixpoint material from CSCE 605 applied to defect detection, and honest about the precision limits.                                                         |
| Google — Building Secure and Reliable Systems, the development chapters (free PDF) | <b>Modules 10 and 12, free in full</b> — and notable for arguing that secure defaults in frameworks beat developer education, which is this course's position too.                                           |
| The CWE and the OWASP Top Ten (free)                                               | <b>The shared vocabulary for Modules 02 through 06.</b> Use CWE identifiers when reporting; it makes a finding searchable and comparable.                                                                    |
| Erickson — Hacking: The Art of Exploitation, 2nd edition                           | <b>Optional, for Module 12's mitigations.</b> Understanding why a mitigation works requires understanding what it stops — <b>and the exercises are for your own machine and the book's own targets only.</b> |

## Module-by-module map

## Module 01 — Why Software Has Vulnerabilities

| Resource                                  | What it covers for this module                                                                                                                            |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dowd, McDonald & Schuh, chapters 1 and 2  | <b>§1 and §2's method</b> — the best treatment of reading code adversarially that exists. Library copy.                                                   |
| The CWE Top 25 and its methodology (free) | <b>§3's severity factors and the evidence for Modules 02 through 06's ordering</b> — including how the ranking is computed, which is worth understanding. |
| CVSS specification (free)                 | <b>§3 as the industry's scoring system</b> — useful, widely required, and read CSCE 701 Module 12 on why a single number misleads.                        |
| MIT 6.858, the first lectures (free, OCW) | <b>§1 and §2 as lectures</b> , free, with the threat model framing developed carefully.                                                                   |

## Module 02 — Memory Safety

| Resource                                            | What it covers for this module                                                                                        |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Dowd, McDonald & Schuh, chapters 5 through 7        | <b>§2 and §3 in depth</b> — the mechanisms explained at the level of what the machine does. Library copy.             |
| Szekeres et al. — SoK: Eternal War in Memory (free) | <b>§1's taxonomy and §3's mitigation history</b> , systematised — the single best overview of this material.          |
| MIT 6.858's buffer overflow lab (free, OCW)         | <b>§2 and §3 practically</b> , on targets built for the purpose — which is Module 01 §4's constraint satisfied.       |
| Google and Microsoft's memory safety reports (free) | <b>§4's evidence</b> — measured proportions of severe defects attributable to memory safety, from codebases at scale. |

## Module 03 — Integers and Types

| Resource                                                           | What it covers for this module                                                                                   |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Dowd, McDonald & Schuh, chapter 6                                  | <b>§1 and §2 exhaustively</b> — the best treatment of C's integer behaviour in a security context. Library copy. |
| Wang et al. — Undefined Behavior: What Happened to My Code? (free) | <b>§2's last point, demonstrated</b> — real security checks deleted by real compilers, with the mechanism.       |
| CERT C Coding Standard, the INT rules (free)                       | <b>§2's practice as a rule set</b> , with compliant and non-compliant examples for each case.                    |
| CWE-190, CWE-680, CWE-843 (free)                                   | <b>§1 and §4 with the shared vocabulary</b> — use these identifiers when reporting.                              |

## Module 04 — Injection and Parsing

| Resource                                               | What it covers for this module                                                                                           |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| OWASP — the injection cheat sheets (free)              | <b>§1 through §3 as current, specific guidance</b> per destination — maintained, and the right reference to hand a team. |
| King — Parse, Don't Validate (free)                    | <b>§4 in the original</b> — short, and it reframes input handling permanently.                                           |
| Dowd, McDonald & Schuh, chapters 8 and 17              | <b>§2 and §3's specifics</b> — encoding confusion and path handling, in more detail than anywhere else. Library copy.    |
| Google's safe-by-construction libraries writeup (free) | <b>§4 at scale</b> — types that make injection unrepresentable, deployed across a large codebase.                        |

## Module 05 — Concurrency Defects

| Resource                                                                          | What it covers for this module                                                                                                       |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Boehm — Threads Cannot Be Implemented As a Library (free)                         | <b>§1's argument for why the memory model must be in the language</b> — and why a data race is undefined rather than merely unlucky. |
| Lu et al. — Learning from Mistakes: a study of real-world concurrency bugs (free) | <b>§2's classification, measured</b> — the study that established atomicity violations as the dominant real-world class.             |
| ThreadSanitizer documentation (free)                                              | <b>§4's primary tool</b> — what it detects, what it misses, and its cost.                                                            |
| CWE-362, CWE-367, CWE-416 (free)                                                  | <b>§2 and §3 with the shared vocabulary</b> — race condition, TOCTTOU, and use-after-free.                                           |

## Module 06 — Deserialisation and Trust Boundaries

| Resource                                                     | What it covers for this module                                                                                                         |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Frohoff & Lawrence — Marshalling Pickles (free)              | <b>§1's gadget chains in the original</b> — the talk that made this class widely understood.                                           |
| OWASP — Deserialization and SSRF cheat sheets (free)         | <b>§2 and §3 as current, per-platform guidance</b> , maintained and specific.                                                          |
| Google — the Java serialization deprecation rationale (free) | <b>§1's severity, as judged by the platform's own maintainers</b> — a language feature deprecated for security reasons, which is rare. |
| CWE-502, CWE-918 (free)                                      | <b>§1 and §3 with the shared vocabulary.</b>                                                                                           |

## Module 07 — Static Analysis

| Resource                                                   | What it covers for this module                                                       |
|------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Møller & Schwartzbach — Static Program Analysis (free PDF) | <b>§1 and §2's theory</b> — free, rigorous, and explicit about the precision limits. |

| Resource                                                 | What it covers for this module                                                                                                                |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| CodeQL documentation and the query examples (free)       | <b>§4's practice</b> — and the query language is worth learning for the project-specific rules alone.                                         |
| Semgrep registry and rule-writing guide (free)           | <b>§4 with a much shallower learning curve</b> — the right starting point for a first custom rule.                                            |
| Bessey et al. — A Few Billion Lines of Code Later (free) | <b>§3's deployment reality</b> , from the people who commercialised it — honest about why adoption fails, and the best thing written on this. |

## Module 08 — Fuzzing

| Resource                                         | What it covers for this module                                                                                                 |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| The Fuzzing Book (free, interactive)             | <b>The whole module, executable</b> — build a coverage-guided fuzzer yourself, which is the fastest route to understanding §1. |
| libFuzzer and AFL++ documentation (free)         | <b>§2 and §3 in practice</b> — the harness rules and the coverage tooling, from the implementations.                           |
| OSS-Fuzz, and its ideal integration guide (free) | <b>§4's continuous operation</b> , plus the best available guidance on harness design at scale.                                |
| Klees et al. — Evaluating Fuzz Testing (free)    | <b>§3's argument, rigorously</b> — why duration and crash counts are poor measures, and what to report instead.                |

## Module 09 — Sanitisers and Runtime Detection

| Resource                                            | What it covers for this module                                                                            |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| LLVM sanitizer documentation (free)                 | <b>§1 and §2's reference</b> — what each detects, what it costs, and which combinations are incompatible. |
| Serebryany et al. — AddressSanitizer (free)         | <b>§1's mechanism in the original</b> — the shadow memory design, which explains the limits.              |
| OSS-Fuzz architecture documentation (free)          | <b>§3's pipeline as deployed</b> — the sanitiser and fuzzer combination at scale.                         |
| Android's HWASan and memory tagging writeups (free) | <b>§4's production detection</b> — the cost figures and the deployment strategy.                          |

## Module 10 — The Secure Development Lifecycle

| Resource                                                                           | What it covers for this module                                                                                 |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Google — Building Secure and Reliable Systems, the development chapters (free PDF) | <b>§1 and §2, free in full</b> — and the safe-by-construction argument is made better here than anywhere else. |

| Resource                                                  | What it covers for this module                                                                             |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Microsoft SDL, and the SAMM maturity model (free)         | <b>§1 as published frameworks</b> — useful for structure, and read §1's callout before adopting wholesale. |
| security.txt (RFC 9116) (free)                            | <b>§4's first step</b> , which takes ten minutes and is still missing from most sites.                     |
| CERT Guide to Coordinated Vulnerability Disclosure (free) | <b>§4 from both sides</b> — the obligations of reporter and maintainer, with the hard cases discussed.     |

## Module 11 — Code Review for Security

| Resource                                          | What it covers for this module                                                                              |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Dowd, McDonald & Schuh, chapters 1 through 4      | <b>§1 and §2's method</b> — the most complete treatment of adversarial code reading in print. Library copy. |
| Google's code review developer guide (free)       | <b>§4's culture</b> , from a very large practice — not security-specific, and the right foundation for it.  |
| OWASP Code Review Guide (free)                    | <b>§1 and §2 as a checklist</b> — useful for completeness, and no substitute for §2's method.               |
| Mozilla and Chromium security review notes (free) | <b>§1's prioritisation in practice</b> — how large projects decide what gets a security review at all.      |

## Module 12 — Exploit Mitigations

| Resource                                                    | What it covers for this module                                                                       |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Szekeres et al. — SoK: Eternal War in Memory (free)         | <b>§1 and §2 systematised</b> — every mitigation, what it stops, and what bypasses it, in one paper. |
| Shacham — The Geometry of Innocent Flesh on the Bone (free) | <b>§2's first bypass in the original</b> — the paper that introduced return-oriented programming.    |
| Debian and Red Hat hardening guides, and checksec (free)    | <b>§3's flags with the distribution's rationale</b> , plus the tool to verify the artefact.          |
| MIT 6.858's mitigation labs (free, OCW)                     | <b>§2 practically</b> , on targets built for it — which is Module 01 §4's constraint satisfied.      |

## Module 13 — Claiming Code Security Honestly

| Resource                                                 | What it covers for this module                                                                                           |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Klees et al. — Evaluating Fuzz Testing (free)            | <b>§1's fuzzing row, rigorously</b> — what a campaign establishes and what is routinely claimed instead.                 |
| Bessey et al. — A Few Billion Lines of Code Later (free) | <b>§1's static analysis row, from the field</b> — honest about what the tools find and what customers believe they find. |

| Resource                                                                          | What it covers for this module                                                                |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Google — Building Secure and Reliable Systems, the concluding chapters (free PDF) | <b>§3's convergence and §4's ordering</b> , argued from operating experience.                 |
| The CWE Top 25, current edition (free)                                            | <b>§4's evidence</b> — reread it annually and check your own programme's emphasis against it. |